

Pet Territory Wars — Milestone 1 Calculator Engine & Simulator v1.0

Backend: Go

Kapsam: Faz 0 — Oyun matematiğinin doğrulanması

Bağımlılık: Milestone 0 tamamlanmış olmalı

Amaç: Mobil uygulama ve gerçek database akışından bağımsız çalışan oyun motorunu oluşturmak

1. Amaç

Bu milestone'un amacı Pet Territory Wars oyununun temel matematik motorunu çalışan Go koduna dönüştürmektir.

Bu aşamada sistem:

- Sahte Walk verileri alabilmeli
- GPS rotasını doğrulayabilmeli
- Geçerli hareket segmentlerini çıkarabilmeli
- Rotayı H3 hücrelerine çevirebilmeli
- Empty capture hesaplayabilmeli
- Owner defense uygulayabilmeli
- Enemy attack uygulayabilmeli
- Dominance decay hesaplayabilmeli
- Ownership transfer üretebilmeli
- Score delta ve domain event üretebilmeli
- Farklı RuleSet değerlerini simülasyonla karşılaştırabilmeli

Bu milestone'da gerçek API, authentication veya production database persistence yapılmayacaktır.

Temel hedef:

Oyun matematiğinin eğlenceli, tutarlı ve teknik olarak uygulanabilir olduğunu doğrulamak.

2. Milestone Sınırı

Yapılacaklar

```
RuleSet
Engine input ve result modelleri
Walk validation
Segment validation
Distance ve speed hesaplama
```

- Route interpolation
- H3 conversion
- Hex visit aggregation
- Dominance decay
- Empty capture
- Owner defense
- Enemy attack
- Ownership transfer
- Score delta
- Domain event
- Determinism
- Replay
- Simulator CLI
- Simulation reports

Yapılmayacaklar

- PostgreSQL repository
- Walk API
- Authentication
- Worker job queue
- Outbox persistence
- Flutter entegrasyonu
- Gerçek leaderboard sorgusu
- NPC
- Guild
- Breed bonus
- Loop bonus
- Push notification

3. Önerilen Paket Yapısı

```
backend/  
├── cmd/  
│   └── simulator/  
│       └── main.go  
├── internal/  
│   ├── gameengine/  
│   │   ├── engine.go  
│   │   ├── input.go  
│   │   ├── result.go  
│   │   ├── rules.go  
│   │   ├── types.go  
│   │   ├── validation.go  
│   │   └── segment.go
```

```

|
|
|   ├── movement.go
|   ├── decay.go
|   ├── territory.go
|   ├── scoring.go
|   ├── events.go
|   └── metadata.go
|
|   ├── infrastructure/
|   │   └── h3adapter/
|   │       ├── resolver.go
|   │       └── resolver_test.go
|   └── simulation/
|       ├── runner.go
|       ├── scenario.go
|       ├── generator.go
|       ├── report.go
|       └── metrics.go
|
└── testdata/
    ├── rules/
    ├── routes/
    └── golden/

```

Paketler gerçek ihtiyaç oluşmadan daha fazla bölünmeyecektir.

4. Engine Public API

Motorun dışarı açacağı temel sözleşme:

```

type Engine struct {
    hexResolver HexResolver
}

func NewEngine(hexResolver HexResolver) *Engine

func (e *Engine) ResolveWalk(
    input ResolveWalkInput,
    rules RuleSet,
) (ResolveWalkResult, error)

```

Motor tek giriş noktasına sahip olacaktır.

Dış katmanlar şu fonksiyonları doğrudan çağırmayacaktır:

```
applyDecay
resolveAttack
resolveCapture
calculateScoreChanges
```

Bunlar package-private kalacaktır.

5. RuleSet Modeli

```
type RuleSet struct {
    Version    string
    Walk       WalkRules
    Movement   MovementRules
    Territory   TerritoryRules
}
```

WalkRules

```
type WalkRules struct {
    MinDurationSeconds int
    MinDistanceMeters  float64
    MinValidRouteRatio float64
    MaxSpeedMPS         float64
    MaxAccuracyMeters   float64
    MaxJumpMeters       float64
}
```

MovementRules

```
type MovementRules struct {
    H3Resolution          int
    MinHexPresenceSeconds int
    MinHexDistanceMeters  float64
    InterpolationMeters   float64
}
```

TerritoryRules

```
type TerritoryRules struct {
    MaxDominance          int
    InitialDominance      int
    OwnerVisitGain        int
}
```

```

    EnemyAttackDamage    int
    DailyDecay            int
    MinimumOwnedDominance int
    ThreatThreshold       int
}

```

6. Başlangıç RuleSet

İlk simülasyon baseline'ı:

```

Version                = mvp-0.1.0
Minimum Walk Duration  = 300 saniye
Minimum Walk Distance  = 300 metre
Minimum Valid Route    = %70
Maximum Speed          = 4.5 m/s
Maximum GPS Accuracy   = 40 metre
Maximum Jump           = 250 metre

H3 Resolution          = aday olarak 8, 9, 10
Minimum Hex Presence   = 8 saniye
Minimum Hex Distance   = 15 metre
Interpolation Distance = 10 metre

Maximum Dominance      = 100
Initial Dominance      = 60
Owner Visit Gain       = 10
Enemy Attack Damage    = 15
Daily Decay            = 5
Minimum Owned Dominance = 1
Threat Threshold       = 30

```

Bu değerler nihai karar değildir.

Simulator bunları karşılaştırmak için kullanılacaktır.

7. Engine Input

```

type ResolveWalkInput struct {
    WalkID      WalkID
    PlayerID    PlayerID
    CityID      CityID

    StartedAt    time.Time
    EndedAt      time.Time
}

```

```

    EvaluatedAt    time.Time

    Route           []LocationPoint
    ExistingHexes   map[HexID]HexState

    Boundary        CityBoundary

    EngineVersion   string
}

```

LocationPoint

```

type LocationPoint struct {
    Sequence      int
    Latitude       float64
    Longitude      float64
    RecordedAt    time.Time
    AccuracyMeters float64
    IsMockLocation bool
}

```

HexState

```

type HexState struct {
    HexID          HexID
    OwnerID        *PlayerID
    Dominance       int
    LastUpdatedAt  time.Time
    Version         int64
}

```

Motor input eksik state'i dış kaynaktan aramayacaktır.

8. Engine Result

```

type ResolveWalkResult struct {
    Status           WalkResolutionStatus
    Validation        ValidationResult
    Metrics           WalkMetrics
    VisitedHexes     []VisitedHex
    HexChanges        []HexChange
    ScoreChanges      []ScoreChange
    Events            []DomainEvent
}

```

```
Metadata      CalculationMetadata
}
```

WalkResolutionStatus

```
RESOLVED
REJECTED
```

Teknik hata `error` olarak döner.

Business rejection normal result'tır.

9. İşlem Sırası

Motor her Walk için aynı sırayı kullanacaktır:

1. RuleSet validation
2. Input validation
3. GPS point validation
4. Segment oluşturma
5. Segment validation
6. City boundary filtering
7. Walk metrics hesaplama
8. Walk validity kararı
9. Route interpolation
10. H3 conversion
11. Hex visit aggregation
12. Decay uygulama
13. Capture / defense / attack
14. Ownership transfer
15. Score delta
16. Domain event
17. Metadata
18. Result

Bu sıra engine contract'ının parçasıdır.

10. Input Validation

Kontroller:

```
Walk ID boş olamaz
Player ID boş olamaz
```

```
StartedAt < EndedAt olmalı  
EvaluatedAt >= EndedAt olmalı  
Route boş olamaz  
Point sequence tekrar edemez  
Timestamp sırası bozuk olamaz  
Latitude geçerli aralıkta olmalı  
Longitude geçerli aralıkta olmalı  
RuleSet geçerli olmalı
```

Teknik olarak bozuk input:

```
error
```

üretir.

Business açısından yetersiz rota:

```
REJECTED
```

üretir.

11. Segment Modeli

```
type RouteSegment struct {  
    From          LocationPoint  
    To            LocationPoint  
    DistanceMeters float64  
    Duration      time.Duration  
    SpeedMPS      float64  
    IsValid       bool  
    InvalidReason SegmentInvalidReason  
}
```

Segment hesaplaması:

```
Distance = Haversine veya güvenilir geo distance  
Duration = To.RecordedAt - From.RecordedAt  
Speed    = Distance / Duration
```

Geçersiz segment nedenleri:

```
NON_POSITIVE_DURATION  
IMPOSSIBLE_SPEED
```


LOCATION_JUMP
LOW_ACCURACY
MOCK_LOCATION
OUTSIDE_CITY

12. Walk Validation

Walk sonucu:

VALID
PARTIALLY_VALID
INVALID

Formül:

Valid Route Ratio =
Valid Segment Distance / Total Segment Distance

Walk geçerli olabilmek için:

Valid Duration >= MinDuration
Valid Distance >= MinDistance
Valid Route Ratio >= MinValidRouteRatio

Tek bir geçersiz segment bütün Walk'ı iptal etmez.

13. Route Interpolation

GPS noktaları seyrek olabilir.

İki geçerli point arasında kalan H3 hücrelerini kaybetmemek için segmentler interpolate edilir.

Başlangıç:

Her 10 metrede bir sanal nokta

Interpolation:

- Gerçek GPS point değildir.
- Validation sinyali üretmez.
- Yalnızca route-to-H3 dönüşümünde kullanılır.

- Orijinal timestamp aralığı içinde sıralı zaman tahmini taşıyabilir.

Interpolation distance RuleSet içindedir.

14. H3 Adapter

Game Engine doğrudan H3 package tiplerine bağımlı olmayacaktır.

```
type HexResolver interface {
    CellFromPoint(
        latitude float64,
        longitude float64,
        resolution int,
    ) (HexID, error)

    Boundary(hexID HexID) ([]GeoPoint, error)
}
```

MVP'de route interpolation engine içinde yapıldığı için adapter yalnızca point-to-cell dönüşümü sağlayabilir.

Hex ID domain tipi:

```
type HexID uint64
```

Persistence dönüşümü sonraki milestone'da ele alınacaktır.

15. Hex Visit Aggregation

Aynı Walk içindeki aynı hex ziyaretleri birleştirilir.

```
type VisitedHex struct {
    HexID          HexID
    PresenceSeconds int
    DistanceMeters float64
    EntryCount     int
    FirstEnteredAt time.Time
    LastExitedAt   time.Time
}
```

Hex etkilenebilir sayılır:

```
PresenceSeconds >= MinHexPresenceSeconds  
OR  
DistanceMeters >= MinHexDistanceMeters
```

Aynı hex aynı Walk içinde yalnızca bir standart oyun etkisi üretir.

İleri geri yürümek sınırsız attack veya defense üretmez.

16. Decay

```
func ApplyDecay(  
    state HexState,  
    evaluatedAt time.Time,  
    rules TerritoryRules,  
) EffectiveHexState
```

Formül:

```
Elapsed Full Days =  
floor(EvaluatedAt - LastUpdatedAt)
```

```
Decay Amount =  
Elapsed Full Days × DailyDecay
```

```
Effective Dominance =  
max(  
    MinimumOwnedDominance,  
    Stored Dominance - Decay Amount  
)
```

Sahipsiz hex:

```
Dominance = 0
```

Pasif decay ownership değiştirmez.

17. Empty Capture

Koşul:

```
OwnerID = nil  
AND  
Visit geçerli
```

Sonuç:

```
New Owner = Current Player  
New Dominance = InitialDominance  
Change Type = EMPTY_CAPTURE
```

Score:

```
Active Hex +1  
Capture +1  
Lifetime Capture +1
```

18. Owner Defense

Koşul:

```
OwnerID = Current Player
```

Formül:

```
New Dominance =  
min(  
    MaxDominance,  
    EffectiveDominance + OwnerVisitGain  
)
```

Aynı Walk içinde aynı hex yalnızca bir defense gain alır.

Dominance zaten maksimumsa `NO_CHANGE` üretilebilir.

19. Enemy Attack

Koşul:

```
OwnerID != nil
AND
OwnerID != Current Player
```

Formül:

```
New Dominance =
max(
    0,
    EffectiveDominance - EnemyAttackDamage
)
```

Dominance sıfırdan büyükse:

```
Owner değişmez
Change Type = ENEMY_ATTACK
```

Dominance tehdit eşiğinin altına inerse:

```
HEX_UNDER_THREAT
```

event'i üretilir.

20. Ownership Transfer

Koşul:

```
Previous Owner != Current Player
AND
Attack sonrası dominance = 0
```

Sonuç:

```
New Owner = Current Player
New Dominance = InitialDominance
Change Type = OWNERSHIP_TRANSFER
```

Score değişimleri:

```
Attacker Active Hex +1
Previous Owner Active Hex -1
```

```
Attacker Steal +1
Attacker Lifetime Steal +1
```

21. HexChange

```
type HexChange struct {
    HexID          HexID
    ExpectedVersion int64

    PreviousOwnerID *PlayerID
    NewOwnerID      *PlayerID

    StoredDominance int
    EffectiveDominance int
    NewDominance    int

    ChangeType HexChangeType
    Reason     HexChangeReason
}
```

Engine database update yapmaz.

`ExpectedVersion`, application katmanının ileride optimistic locking uygulaması içindir.

22. ScoreChange

```
type ScoreChange struct {
    PlayerID PlayerID
    Metric   ScoreMetric
    Delta    int64
    Reason   ScoreChangeReason
    WalkID   WalkID
}
```

Score sonucu toplam değeri değil delta olarak üretilir.

Örnek:

```
ACTIVE_HEX +1
CAPTURE +1
LIFETIME_CAPTURE +1
```

Bu yaklaşım sonraki database milestone'unda atomic increment için uygundur.

23. Domain Events

MVP event türleri:

```
WALK_VALIDATED
WALK_PARTIALLY_VALIDATED
WALK_REJECTED
EMPTY_HEX_CAPTURED
HEX_DEFENDED
HEX_ATTACKED
HEX_UNDER_THREAT
HEX_CAPTURED_FROM_RIVAL
HEX_LOST
PLAYER_SCORE_CHANGED
```

Engine yalnızca event listesi üretir.

Event publish etmez.

24. Determinism

Motor şu kurallara uymalıdır:

```
time.Now kullanılmaz
Random kullanılmaz
Global mutable state kullanılmaz
Database erişimi yapılmaz
HTTP çağrısı yapılmaz
Map iteration sırasına güvenilmez
```

Hex işlemleri:

```
HexID ASC
```

sırasıyla yürütülmelidir.

Aynı input ve RuleSet aynı sıralı sonucu üretmelidir.

25. Engine Metadata

```
type CalculationMetadata struct {  
    EngineVersion string  
    RuleSetVersion string  
    InputHash      string  
    EvaluatedAt    time.Time  
}
```

Input hash şunları kapsar:

```
Walk ID  
Player ID  
Ordered GPS points  
Existing hex states  
RuleSet version  
Evaluation time
```

Hash yöntemi:

```
SHA-256
```

26. Simulator Amacı

Simulator production engine'i kullanacaktır.

Ayrı oyun matematiği yazılmayacaktır.

Simulator'ın görevi:

```
RuleSet üretmek  
Sahte rota üretmek  
Sahte oyuncu üretmek  
Sahte territory state üretmek  
Engine'i tekrar tekrar çağırmak  
Sonuçları raporlamak
```

27. Simulator CLI

Örnek kullanım:


```
go run ./cmd/simulator
  --scenario=density
  --players=100
  --days=30
  --resolution=9
  --seed=42
```

CLI parametreleri:

```
scenario
players
days
walks-per-day
resolution
attack-damage
daily-decay
initial-dominance
seed
output
```

Simulator random kullanabilir.

Ancak seed zorunlu olmalıdır.

Aynı seed aynı simülasyonu üretmelidir.

Engine'in kendisi random kullanmaz.

28. Simulator Senaryoları

Resolution Comparison

```
H3 8
H3 9
H3 10
```

ölçülür.

Density

```
10 oyuncu
50 oyuncu
200 oyuncu
```

aynı bölgede test edilir.

Dominance

Initial: 50 / 60 / 70
Attack: 10 / 15 / 20
Decay: 3 / 5 / 8

karşılaştırılır.

Repeated Route

Oyuncuların aynı mahalle rotasını her gün yürümesi test edilir.

Rival Conflict

Birden fazla oyuncu aynı hex'lere saldırır.

GPS Noise

Rotaya kontrollü GPS jitter eklenir.

Power User

Bir oyuncu diğerlerinden çok daha fazla Walk yapar.

MVP'de profesyonel gezdirici modu yoktur; yalnızca dengesizlik riski ölçülür.

29. Simulator Metrikleri

Walk başına unique visited hex
Walk başına affected hex
Empty capture sayısı
Attack sayısı
Ownership transfer sayısı
Ortalama dominance
Ortalama hex yaşam süresi
Oyuncu başına active hex
En güçlü oyuncu payı
Harita sahipsiz oranı
Harita kilitlenme oranı
Yeni oyuncu ilk Walk başarısı
Rejected Walk oranı
Partial Walk oranı

30. Simulation Report

Console özeti:

```
Scenario: resolution-comparison
Players: 100
Days: 30
RuleSet: mvp-0.1.0

Average affected hexes per walk: 18.4
Ownership transfers per day: 37
First-walk capture success: 94%
Top player territory share: 8.2%
Rejected walks: 3.1%
```

Machine-readable çıktı:

```
JSON
CSV
```

MVP için JSON yeterlidir.

31. Test Stratejisi

Unit Tests

```
RuleSet validation
Distance
Speed
Segment validation
Walk validation
Interpolation
Hex aggregation
Decay
Empty capture
Defense
Attack
Transfer
Score delta
Event generation
```

Golden Tests

Empty capture Walk
Owner defense Walk
Enemy attack Walk
Ownership transfer Walk
Partial invalid Walk
Fully rejected Walk

Replay Tests

Aynı input yüzlerce kez çalıştırılır.

Sonuç birebir aynı olmalıdır.

Property Tests

Mümkün olan invariant'lar:

Dominance negatif olamaz
Dominance maksimumu aşamaz
Sahipsiz hex dominance 0 olmalı
Owner transfer yalnızca attack sonucu oluşmalı
Score delta beklenmeyen oyuncuya gitmemeli

32. Benchmark

Öncelikli benchmark'lar:

ResolveWalk – 100 point
ResolveWalk – 1.000 point
ResolveWalk – 10.000 point
Route interpolation
H3 conversion
Hex aggregation

Hedef kesin performans garantisi değildir.

Ama regression görülmelidir.

Örnek başlangıç hedefi:

```
Normal 30 dakikalık Walk:  
Engine calculation < 100 ms
```

Bu hedef donanım ve rota büyüklüğüne göre değerlendirilir.

33. Error Model

Business result:

```
Walk rejected  
Segment invalid  
Outside city  
Too short
```

normal result içinde döner.

Technical error:

```
Invalid RuleSet  
H3 resolver failure  
Malformed boundary  
Unsupported resolution  
Invariant violation
```

Go error olarak döner.

Typed error önerilir:

```
type EngineError struct {  
    Code EngineErrorCode  
    Cause error  
}
```

34. İlk Implementasyon Sırası

1. Domain ID ve value type'ları
2. RuleSet modelleri
3. RuleSet validation
4. Input ve result modelleri
5. Distance ve segment hesapları
6. Walk validation

7. H3 adapter
8. Route interpolation
9. Hex visit aggregation
10. Decay
11. Empty capture
12. Owner defense
13. Enemy attack
14. Ownership transfer
15. Score changes
16. Domain events
17. ResolveWalk orchestration
18. Golden tests
19. Simulator runner
20. Simulation reports

35. Calculator Rules ile Uyum

Engine yalnızca mevcut MVP Calculator Rules'u uygular.

Bu milestone sırasında yeni mekanik eklenmez.

Eklenmeyecekler:

Loop bonus
Clan
Breed bonus
NPC
Booster
Sponsored hex
Dynamic resolution

Simülasyonda sorun görülürse Calculator Rules değiştirilmeden önce sonuç birlikte değerlendirilir.

Yeni karar verilirse yeni RuleSet sürümü oluşturulur.

36. Definition of Done

Milestone 1 tamamlanmış sayılırsa:

1. Engine tek ResolveWalk girişine sahip.
2. Engine stateless.
3. Engine deterministik.
4. RuleSet validation çalışıyor.
5. Walk validation çalışıyor.

6. Geçersiz segmentler ayrıştırılıyor.
7. Route H3 hücrelerine çevriliyor.
8. Aynı hex ziyaretleri birleştiriliyor.
9. Decay çalışıyor.
10. Empty capture çalışıyor.
11. Owner defense çalışıyor.
12. Enemy attack çalışıyor.
13. Ownership transfer çalışıyor.
14. Score delta üretiliyor.
15. Domain event üretiliyor.
16. Golden testler geçiyor.
17. Replay testi geçiyor.
18. Resolution 8/9/10 simüle edilebiliyor.
19. Simulator JSON rapor üretiyor.
20. Engine database veya HTTP bağımlılığı taşımıyor.

37. Milestone Sonucu

Milestone 1 sonunda şu sorulara ölçülebilir cevap verilmelidir:

H3 resolution 9 mu 10 mu daha uygun?
Ortalama Walk kaç hex etkiliyor?
Initial dominance 60 dengeli mi?
Attack damage 15 yeterli mi?
Daily decay 5 çok mu yavaş?
Yeni oyuncu ilk Walk'ta başarı hissediyor mu?
Yoğun mahallede ownership ne sıklıkta değişiyor?
Power user oyunu domine ediyor mu?
GPS jitter sonucu bozuyor mu?

Bu sonuçlar görülmeden database ve API implementasyonunda oyun matematiği sabit kabul edilmez.

38. Nihai Karar

Milestone 1'in çıktısı:

Saf Go Game Engine
Versiyonlu RuleSet
Deterministik Walk çözümleme
H3 tabanlı territory sonucu
Test edilebilir Calculator Rules
Tekrarlanabilir simulator
Ölçülebilir denge raporu

olacaktır.

Bu milestone tamamlandıktan sonra sıradaki aşama:

Milestone 2 — Core Database Implementasyonu